

Emergency Mesh: Technical Reference

Team LVISG: Manit Bhasin, Divit Rastogi, Rajveer Kapoor

BUILT

10 October 2026

COMMIT

6946197

GENERATOR

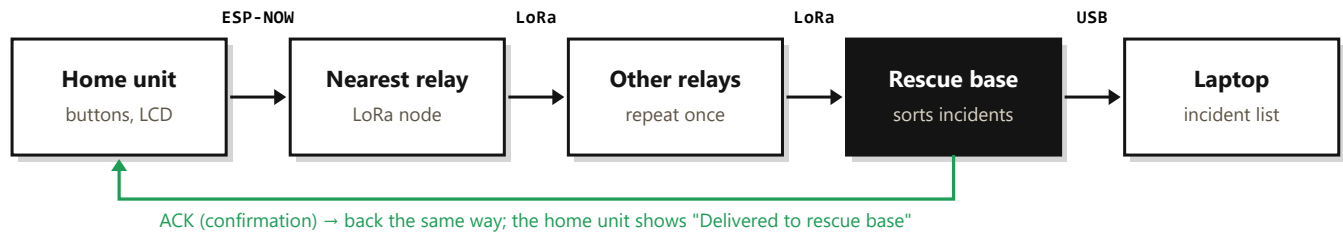
docs/tools/build_technical_reference.py

Every number in this document is read from the repository (code, results and README) or computed from it by the generator. Facts from outside sources are listed in section 10. Where text is quoted from the README, the section numbers in it are README sections.

2	System at a glance	8	Results
3	Packet format	9	Duty-cycle check
4	Time on air	10	Outside facts
5	Radio settings	11	Wokwi prototype pins
6	Node rules and forwarding	12	Known limits
7	Simulator setup		

2 System at a glance

A person presses a button on their home unit. The SOS goes over ESP-NOW to the nearest relay node, then relay to relay over LoRa to the rescue base. The base confirms with an ACK that travels back the same way.



Path of one SOS: home unit → nearest relay → other relays → rescue base → laptop. LoRa between relays at SF9, 125 kHz (`RadioConfig`); every relay repeats each new message once.

What is built and what is designed only

Part	Status	Where
Mesh logic: message IDs, duplicate dropping, hop limits, priority queue, random delays, listen-before-talk, delivery confirmations, retries, rate limits, heartbeats	Built and tested in a Python network simulator (16 automated tests)	<code>emergency-mesh/simulation/</code>
Disaster scenarios: rerouting, node failures, mass SOS, dead-node detection, sensitivity to radio assumptions	Built , results in Section 3	<code>emergency-mesh/simulation/results/</code>
Packet format (max 31 bytes) and LoRa time-on-air	Built	<code>protocol.py</code>
Node interface: OLED screen, SOS button, status LED	Built in the Wokwi ESP32 simulator (MicroPython)	<code>emergency-mesh/wokwi_node/</code> , live project
Interactive map demo: click anywhere to send an SOS, destroy or repair nodes, simulate a crowd	Built , runs in any web browser	<code>emergency-mesh/web_demo/</code>
Rescue dashboard	Prototype : priority-sorted incident table from the simulator (also saved as CSV)	<code>python run.py demo</code>
ESP32 firmware with LoRa radio	Designed , not built	Sections 5–6
Home unit (ESP-NOW)	Designed , not built	Sections 6 and 7
Schematics: system diagram, node block diagram, Wokwi circuit	Included	Sections 2 and 6
Physical nodes	Not done yet	Section 10
Field range tests for LoRa and ESP-NOW	Not done yet	Section 10

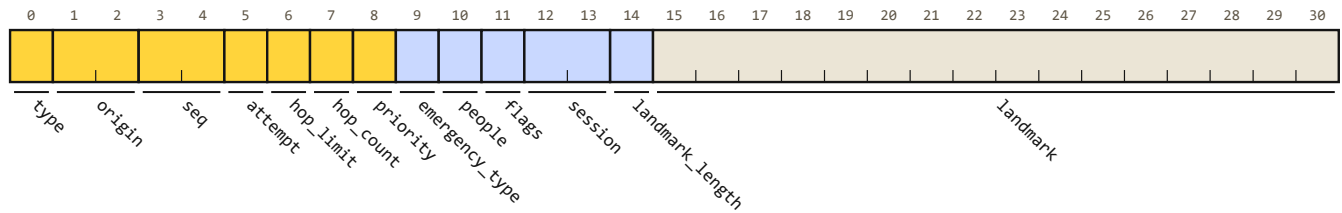
From the README, "Project status". Section numbers in this table are README sections.

3

Packet format

All three packet types share a 9-byte header. Multi-byte fields are little-endian (`struct` format prefix `<`), so the low byte is sent first. Layouts are read from `protocol.py`.

SOS packet, bytes 0–30 (15–31 bytes)

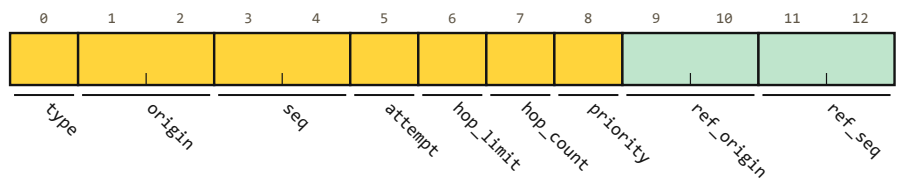


Header
 SOS body
 Landmark (optional)

Bytes	Field	Size	Type	Meaning
0	type	1 B	uint8	SOS, ACK or HEARTBEAT
1–2	origin	2 B	uint16, little-endian	Node where the message was created
3–4	seq	2 B	uint16, little-endian	Counter on that node; (origin, seq) identifies the emergency
5	attempt	1 B	uint8	1 for the first send, 2+ for retries
6	hop_limit	1 B	uint8	Starts at 6, drops by 1 per hop
7	hop_count	1 B	uint8	Hops travelled so far
8	priority	1 B	uint8	3 critical, 2 urgent, 1 supplies, 0 "I'm safe"
9	emergency_type	1 B	uint8	Medical, trapped, flooding, fire, collapse, other
10	people	1 B	uint8	Number of people
11	flags	1 B	uint8	Injured, trapped, child, elderly, water rising
12–13	session	2 B	uint16, little-endian	Sender ID: the home unit's ID, used for rate limiting and to look up the address registered at install
14	landmark_length	1 B	uint8	Number of landmark bytes that follow
15–30	landmark	0–16 B	UTF-8 text	e.g. "Blue gate, Ln 3" (plus 1 length byte)

Header `struct.Struct("<BHHBBBB")` = 9 B; SOS body `struct.Struct("<BBBHB")` = 6 B; landmark $\leq$ 16 B (`MAX_LANDMARK_BYTES`), cut so a UTF-8 character is never split. Meanings from README section 5.

ACK packet (13 bytes)

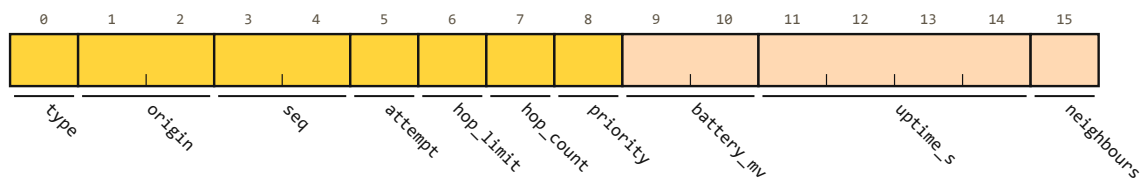


Header ACK body

Bytes	Field	Size	Type	Meaning
9–10	ref_origin	2 B	uint16, little-endian	origin of the SOS being confirmed
11–12	ref_seq	2 B	uint16, little-endian	seq of the SOS being confirmed

Bytes 0–8 are the same header as the SOS packet.

Heartbeat packet (16 bytes)



Header Heartbeat body

Bytes	Field	Size	Type	Meaning
9–10	battery_mv	2 B	uint16, little-endian	Battery voltage in millivolts
11–14	uptime_s	4 B	uint32, little-endian	Seconds since the node started
15	neighbours	1 B	uint8	Nodes within radio range

Codes

Code	type
1	SOS
2	ACK
3	HEARTBEAT

Code	emergency_type
1	MEDICAL
2	TRAPPED
3	FLOODING
4	FIRE
5	COLLAPSE
6	OTHER

Code	priority	Meaning
3	CRITICAL	life-threatening: injured, trapped, medical
2	URGENT	water rising, children or elderly at risk
1	SUPPLIES	food, water, medicine needed
0	SAFE	"I'm safe" check-in

Value	Bit	flags
1	bit 0	INJURED
2	bit 1	TRAPPED
4	bit 2	CHILD
8	bit 3	ELDERLY
16	bit 4	WATER_RISING

Flags combine by adding bits: INJURED|TRAPPED = 3.

Worked example: encode() of the first SOS in the rerouting demo

The critical SOS from node 1 in run.py demo (seed 7): trapped, 4 people, flags INJURED|TRAPPED, landmark "School rooftop". It encodes to 29 bytes:

```
01 01 00 01 00 01 06 00 03 02 04 03 65 00 0e 53 63 68 6f 6f 6c 20 72 6f 6f 66 74 6f 70
```

Bytes	Field	Hex	Value
0	type	01	1 (SOS)
1-2	origin	01 00	1
3-4	seq	01 00	1
5	attempt	01	1
6	hop_limit	06	6
7	hop_count	00	0
8	priority	03	3 (CRITICAL)
9	emergency_type	02	2 (TRAPPED)
10	people	04	4
11	flags	03	3 (INJURED TRAPPED)
12-13	session	65 00	101
14	landmark_length	0e	14
15-28	landmark	53 63 68 6f 6f 6c 20 72 6f 6f 66 74 6f 70	"School rooftop"

Built by calling encode() on the packet the simulator created; decode() gives back the same packet. It matches row 1 of results/demo_dashboard.csv.

4 Time on air

Each packet's time on air comes from `lora_airtime()` in `protocol.py`, the SX127x formula published by Semtech. Settings: SF9, 125 kHz, coding rate 4/5, 8-symbol preamble, CRC on, explicit header.

Our packets at SF9

Packet	Size	Payload symbols	Time on air
ACK	13 B	28	164.9 ms
SOS, no landmark	15 B	28	164.9 ms
Heartbeat	16 B	28	164.9 ms
SOS, longest landmark	31 B	48	246.8 ms

The formula, as coded

```
t_sym = (2 ** spreading_factor) / bandwidth_hz
low_dr = 1 if t_sym > 0.016 else 0
ih = 0 if explicit_header else 1
numerator = 8 * payload_bytes - 4 * spreading_factor + 28 + 16 * int(crc) - 20 * ih
payload_symbols = 8 + max(
    math.ceil(numerator / (4 * (spreading_factor - 2 * low_dr))) * (coding_rate + 4), 0)
)
return (preamble_symbols + 4.25) * t_sym + payload_symbols * t_sym
```

Worked example: a 31-byte SOS at SF9

Step	Calculation	Result
Symbol time	$2^9 / 125,000 \text{ Hz}$	4.096 ms
Low data rate optimisation	on only if a symbol is longer than 16 ms	off
Numerator	$8 \times 31 - 4 \times 9 + 28 + 16 \times 1 - 20 \times 0$	256
Blocks	$\lceil 256 / (4 \times (9 - 2 \times 0)) \rceil = \lceil 7.11 \rceil$	8
Payload symbols	$8 + 8 \times (1 + 4)$	48
Preamble symbols	$8 + 4.25$	12.25
Time on air	$(12.25 + 48) \times 4.096 \text{ ms}$	246.8 ms

Why several sizes take the same time

The payload is sent in blocks: each block carries 36 bits ($4 \times \text{SF9}$) and costs 5 symbols, and a part-filled block costs the same as a full one. So time on air rises in steps:

Packet size	Payload symbols	Time on air
13–17 B	28	164.9 ms
18–21 B	33	185.3 ms
22–26 B	38	205.8 ms
27–30 B	43	226.3 ms
31 B	48	246.8 ms

The ACK (13 B) and heartbeat (16 B) fall in the same step as an SOS with no landmark (15 B).

Spreading factor

Spreading factor	Symbol time	Low data rate opt.	ACK (13 B)	SOS (31 B)
SF7	1.024 ms	off	46.3 ms	71.9 ms
SF8	2.048 ms	off	82.4 ms	133.6 ms
SF9	4.096 ms	off	164.9 ms	246.8 ms
SF10	8.192 ms	off	288.8 ms	452.6 ms
SF11	16.384 ms	on	577.5 ms	905.2 ms
SF12	32.768 ms	on	1155.1 ms	1810.4 ms

SF7–SF12 is the range `run.py --sf` accepts; all results use SF9.

5 Radio settings

Assumed, not measured. No radio has been tested in the field yet. The range and loss figures come from published sources and our own allowances. README section 3.5 shows how much the results depend on them.

In the code

Setting	Value	Where
Spreading factor	SF9	<code>RadioConfig.spreading_factor</code>
Bandwidth	125 kHz	<code>RadioConfig.bandwidth_hz</code>
Coding rate	4/5	<code>lora_airtime(coding_rate=...)</code>
Preamble	8 symbols	<code>lora_airtime(preamble_symbols=...)</code>
Range	800 m	<code>RadioConfig.range_m</code>
Random packet loss	2%	<code>RadioConfig.loss_probability</code>

Why these values (README section 4)

Value	Reason
865–868 MHz band	Licence-exempt in India under the Use of Low Power Equipment in the Frequency Band 865–868 MHz for Short Range Devices (Exemption from Licence) Rules, 2021 (G.S.R. 853(E)). For tracking, tracing and data acquisition devices: at most 500 mW e.r.p., channels up to 200 kHz, transmitting at most 2.5% of the time (10% for network access points), adaptive power control, and type-approved equipment
866.0 MHz	Inside that band, with room on both sides for a 125 kHz channel
ESP-NOW on 2.4 GHz (home unit to nearest relay)	Licence-exempt in India
SF9, 125 kHz, coding rate 4/5	SF9 is the suggested starting point for Indian outdoor projects; 125 kHz and 4/5 are the standard settings (Zbotic SX1276 guide)
800 m range	The same guide lists 500 m–1 km at SF9 in dense Indian cities (Delhi, Mumbai, Bangalore) for an SX1276 at +20 dBm with simple antennas. 800 m sits inside that range; Section 3.5 tests 500–1000 m
+20 dBm transmit power	The SX1276's maximum (100 mW), which the range figure assumes; under the 500 mW e.r.p. limit with a simple antenna
200 m home unit reach (interactive demo, assumed)	Our assumption for the demo. A published ESP-NOW test (Espressif developer blog, "ESP-NOW for outdoor applications", by a community author) found about 150 m reliable with built-in antennas, about 60% delivery at 300 m in an open field, and about 100% to 450 m in long-range mode. Not measured by us
0.25 s per SOS on air	Calculated with the time-on-air formula in Semtech's SX1276 datasheet, for a 31-byte packet at SF9

6 Node rules and forwarding

Every `NodeConfig` field, its value in `node.py`, and the reason from README section 4. The generator checks that each value matches the README.

In the code	README	Why
<code>hop_limit = 6</code>	Hop limit 6	Farthest node is 4 hops away; 2 spare hops allow detours around destroyed nodes
<code>seen_cache_size = 256</code>	Remember last 256 message IDs	Would use about 1.5 KB on the ESP32. A 16× larger memory didn't change the heavy-load results, so 256 is enough
<code>ack_timeout_s = 90</code> <code>max_attempts = 5</code>	Retry after 90 s, up to 5 sends	Normal delivery takes seconds (median 2.5–12.5 s in Section 3.2), so a retry means the message was truly lost. Waits double each time, so the last retry is about 23 minutes after the first send
<code>jitter_max_s = 1</code> <code>busy_backoff_s = (0.2, 1.5)</code>	Wait up to 1 s before sending; 0.2–1.5 s if busy	Spreads transmissions over several packet lengths (one SOS ≈ 0.25 s) to reduce collisions
<code>rate_limit_count = 3</code> <code>rate_limit_window_s = 600</code>	3 SOS per home unit per 10 minutes	Lets someone update their SOS but stops one unit flooding the network
<code>heartbeat_interval_s = 900</code> <code>silent_alert_s = 1800</code>	Heartbeat 15 min, alert at 30 min	An alert needs two missed heartbeats, so one lost packet doesn't cause a false alarm

Forwarding rules (README section 5)

1. Drop any message ID already seen.
2. Forward only while `hop_limit > 1`.
3. Always send the highest-priority waiting packet first.
4. Wait a random delay before sending; back off if the channel is busy.
5. Copies travel along every surviving path, so there is no route to repair when a node dies.
6. The base confirms every SOS attempt it receives.
7. Without a confirmation, the sender retries with doubling waits, up to 5 sends.
8. Rate limit per sender ID (one per home unit); heartbeats from every node.

7 Simulator setup

`grid_nodes()` places $5 \times 5 = 25$ nodes, 600 m apart with a random offset of up to ± 100 m each way, over $2.4 \text{ km} \times 2.4 \text{ km}$. The centre point is the rescue base, so there are **24 relays + 1 base**.

In the seed-7 demo network, each relay has 2–6 neighbours within 800 m (mean 3.9), and the farthest relay is 4 hops from the base, inside the hop limit of 6.

Radio model (from `network.py`)

- A node hears every live node within `range_m` (flat ground, no terrain).
- Two transmissions overlapping at a receiver destroy each other (collision).
- A node cannot receive while it is transmitting (half-duplex).
- Each successful reception can still be lost at random (`loss_probability`).

Scenarios, as run by `python run.py all`

Scenario	What happens	Runs
Rerouting demo	A critical SOS from the farthest node plus 4 random SOS; the relay in the middle of its route is then destroyed and a second SOS is sent from the same node.	seed 7
Failures	Destroy 0%, 10%, 20%, 30%, 40%, 50% of relays at random, then every surviving node sends one SOS within 30 s. Run for 30 min.	20 networks per row, seed 7
Load	10, 50, 100, 200, 500 SOS from random nodes within 1 min, no nodes destroyed. Run for 1 h. Priority mix: critical 30%, urgent 30%, supplies 20%, safe 20%.	5 networks per row (<code>-- trials // 4</code>), seed 7
Health	Every relay sends a heartbeat every 15 min; 3 nodes die silently after 1 h; the base checks at 3 h and flags nodes silent for over 30 min.	10 runs (<code>-- trials // 2</code>), seed 7
Sensitivity	The failure test at ranges 500 m, 650 m, 800 m, 1000 m and random loss 0%, 2%, 5%, 10%, with 0%, 30% of relays destroyed.	20 networks per row, seed 7

Defaults read from `scenarios.py` and `run.py` (`-- trials 20`, `-- seed 7`). The README reports that seeds 3 and 11 gave the same main conclusions.

Why this layout (README section 4)

Value	Reason
25 nodes, 2.4 km × 2.4 km	Roughly neighbourhood-sized
600 m spacing, ±100 m random offset	Below the 800 m range, so each node normally reaches its nearest neighbours but rarely its diagonal ones (about 850 m away). This forces multi-hop routes. The offset stops the grid being perfectly regular
Base at the centre	Farthest nodes are 4 hops away
2% random loss	Allowance for interference the model doesn't cover; Section 3.5 shows results hardly change from 0% to 10%

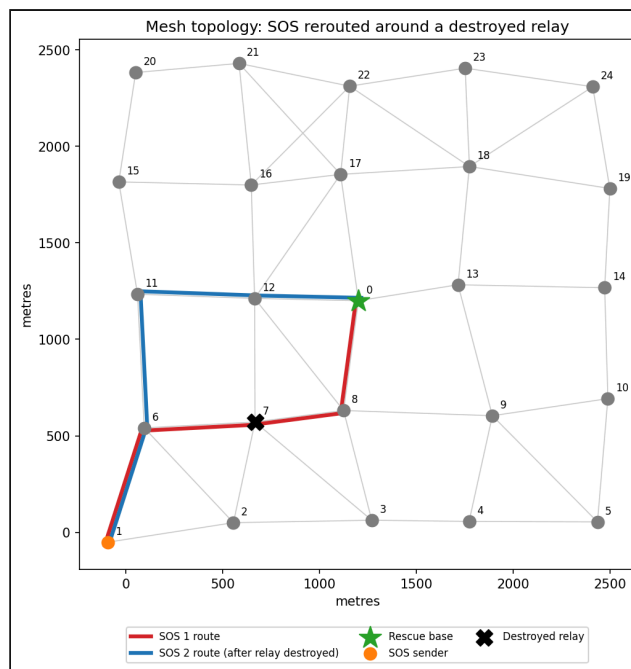
8 Results

Every CSV in `emergency-mesh/simulation/results/`, with every column. Column names are shown under each heading.

Rerouting demo: rescue dashboard

Priority priority	Type type	Node node	Latitude lat	Longitude lon	People people	Flags flags	Landmark landmark	Hops hops	Delay (s) delay_s
CRITICAL	TRAPPED	1	12.98104	80.21713	4	INJURED TRAPPED	School rooftop	4	2.9
CRITICAL	MEDICAL	11	12.99258	80.21855	7	INJURED TRAPPED	Water tank	2	1.5
URGENT	FIRE	16	12.99767	80.22396	1	CHILD	Water tank	2	0.8
URGENT	FLOODING	17	12.99816	80.22821	8	CHILD	Water tank	1	1.1
URGENT	FLOODING	1	12.98104	80.21713	6	ELDERLY WATER_RISING	School rooftop	4	1.9
SAFE	OTHER	18	12.99853	80.23434	5	-	Water tank	2	6.5

From `emergency-mesh/simulation/results/demo_dashboard.csv`; 6 rows, every column.

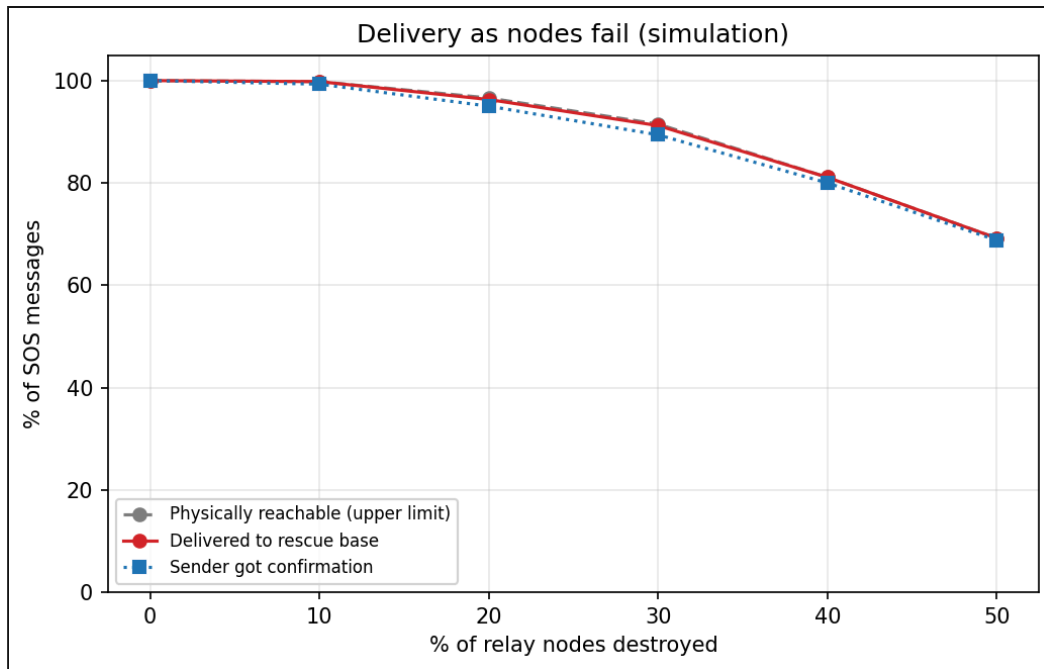


`results/demo_topology.png`: the demo network, both SOS routes and the destroyed relay

Delivery as relays are destroyed

Relays destroyed (%) failed_pct	SOS sent sos_snt	Reachable (%) reachable_pct	Delivered (%) delivered_pct	Delivered of reachable (%) delivered_of_reachable_pct	Sender confirmed (%) confirmed_pct	Median delay (s) median_delay_s	95th percentile delay (s) p95_delay_s	Mean hops mean_hops	Transmissions per SOS tx_per_sos
0	480	100.0	100.0	100.0	100.0	12.5	289.7	3.11	45.2
10	440	99.8	99.8	100.0	99.3	8.7	291.0	3.03	43.0
20	380	96.6	96.3	99.7	95.0	6.9	288.9	2.9	35.8
30	340	91.5	91.2	99.7	89.4	4.7	288.3	2.67	30.0
40	280	81.1	81.1	100.0	80.0	3.5	279.1	2.44	21.3
50	240	69.2	69.2	100.0	68.8	2.5	280.1	2.39	18.6

From `emergency-mesh/simulation/results/failures.csv`; 6 rows, every column.

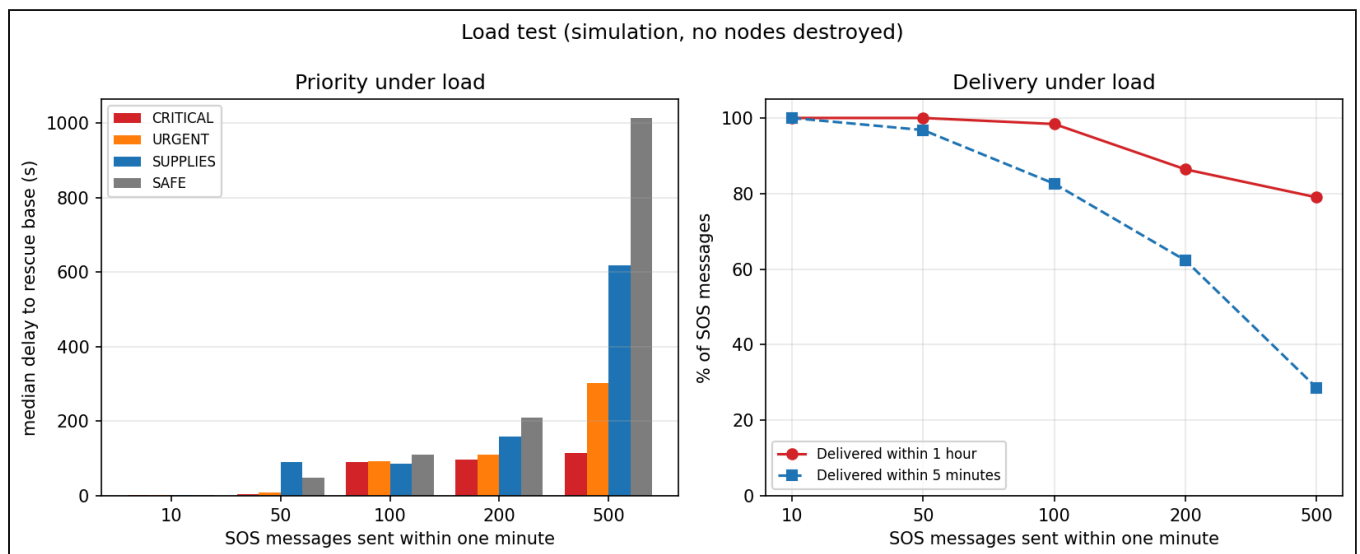


results/failures.png: delivery as relays are destroyed

Many SOS at once

SOS in 60 s sos_in_60s	Delivered (%) delivered_pct	Delivered within 5 min (%) within_5min_pct	Median delay, critical (s) median_delay_critical_s	Median delay, urgent (s) median_delay_urgent_s	Median delay, supplies (s) median_delay_supplies_s	Median delay, "I'm safe" (s) median_delay_safe_s	95th percentile delay (s) p95_delay_s	Collisions per SOS collisions_per_sos
10	100.0	100.0	2.8	2.4	1.9	1.5	91.4	22.8
50	100.0	96.8	5.5	8.8	91.1	49.4	290.9	33.5
100	98.4	82.6	90.8	92.7	85.8	110.4	684.7	41.5
200	86.4	62.3	97.5	110.5	159.9	209.4	1387.2	40.3
500	79.0	28.6	114.4	302.2	618.5	1013.7	1429.0	50.3

From emergency-mesh/simulation/results/load.csv; 5 rows, every column.



results/load.png: delivery and delay under load

Detecting dead nodes

Runs trials	Nodes killed nodes_killed	Detected detected	Missed missed	False alarms false_alarms	Mean time to alert (min) mean_minutes_to_alert	Longest time to alert (min) max_minutes_to_alert
10	30	30	0	0	20.8	28.0

From `emergency-mesh/simulation/results/health.csv`; 1 row, every column.

Sensitivity to the radio assumptions

Range (m) range_m	Random loss (%) loss_pct	Relays destroyed (%) failed_pct	Reachable (%) reachable_pct	Delivered (%) delivered_pct	Delivered of reachable (%) delivered_of_reachable_pct	Median delay (s) median_delay_s
500	2	0	0.0	0.0	n/a	n/a
500	2	30	0.0	0.0	n/a	n/a
650	2	0	82.7	82.3	99.5	6.7
650	2	30	52.1	52.1	100.0	2.7
800	2	0	100.0	100.0	100.0	12.5
800	2	30	91.5	91.2	99.7	4.7
1000	2	0	100.0	100.0	100.0	13.6
1000	2	30	99.7	99.7	100.0	6.2
800	0	0	100.0	100.0	100.0	9.9
800	0	30	91.5	91.5	100.0	4.8
800	5	0	100.0	100.0	100.0	11.2
800	5	30	91.5	91.5	100.0	6.1
800	10	0	100.0	100.0	100.0	13.1
800	10	30	91.5	91.2	99.7	5.3

From `emergency-mesh/simulation/results/sensitivity.csv`; 14 rows, every column.

9 Duty-cycle check (our estimate)

Our own estimate, not a measurement. Under India's rules for the 865–868 MHz band (README section 4), each relay may transmit at most 2.5% of the time. We assume a 1-hour observation period. Because every relay repeats every message, each relay's load is the whole network's traffic.

Quantity	Calculation	Result
Heartbeat size and time on air	16 B at SF9	164.9 ms
Heartbeats per relay per hour	$3600 \text{ s} / 900 \text{ s}$	4
Heartbeats each relay sends per hour	24 relays $\times$ 4: its own, plus each other relay's once	96
Heartbeat time per relay per hour	$96 \times 164.9 \text{ ms}$	$15.8 \text{ s} = \mathbf{0.44\%}$
Budget at the limit	2.5% of 3600 s	90 s
Left for SOS traffic	$90 \text{ s} - 15.8 \text{ s}$	74.2 s
Time per SOS at each relay	SOS 31 B (246.8 ms) + its ACK 13 B (164.9 ms)	411.6 ms
SOS per hour before the limit	$74.2 \text{ s} / 411.6 \text{ ms}$	about 180 , across the whole network
The 500-SOS load test	$15.8 \text{ s} + 500 \times 411.6 \text{ ms}$	$222 \text{ s} = \mathbf{6.2\%}$ of the hour

- Assumes each relay forwards each SOS and its ACK exactly once (duplicates are dropped), with no retries. Retries in a real mass event only add to this.
- Heartbeats alone use 0.44% of each relay's time. The 500-SOS load test sends 500 SOS within 1 min, about 2.8 times what fits in a whole hour under the limit.
- Home units use ESP-NOW on 2.4 GHz, a different band, so they don't count against this limit.
- Gradient routing (README section 10) would cut how many relays repeat each message.

10 Outside facts

Facts this design relies on that come from outside the repository. None of them were measured by us. The list lives in `docs/tools/outside_facts.csv`.

Fact	Source	Link
865–868 MHz is licence-exempt for short range devices. For tracking, tracing and data acquisition devices: at most 500 mW e.r.p., channels up to 200 kHz, transmitting at most 2.5% of the time (10% for network access points), adaptive power control, and type-approved equipment.	DoT, Use of Low Power Equipment in the Frequency Band 865–868 MHz for Short Range Devices (Exemption from Licence) Rules, 2021 (G.S.R. 853(E))	<i>not linked yet</i>
The 2.4 GHz band used by ESP-NOW is licence-exempt in India.	DoT notification G.S.R. 45(E), 2005	<i>not linked yet</i>
The SX1276's maximum transmit power is +20 dBm (100 mW). Time on air follows Semtech's published formula, used in <code>protocol.py</code> .	Semtech SX1276 datasheet	<i>not linked yet</i>
ESP-NOW carries up to 250 bytes per packet (v1) or 1470 bytes (v2). At most 20 paired peers, 17 of them encrypted. Broadcast and unencrypted packets are received without pairing. Encryption uses CCMP.	Espressif ESP-NOW documentation	<i>not linked yet</i>
ESP-NOW with built-in antennas: about 150 m reliable, about 60% delivery at 300 m in an open field, and about 100% to 450 m in long-range mode. Not measured by us.	Espressif developer blog, "ESP-NOW for outdoor applications" (community author)	<i>not linked yet</i>
HD44780 LCD controllers store only 8 custom characters.	HD44780 datasheet	<i>not linked yet</i>
500 m–1 km range at SF9 in dense Indian cities (Delhi, Mumbai, Bangalore) for an SX1276 at +20 dBm with simple antennas. Our assumed 800 m range sits inside it.	Zbotic, SX1276 LoRa module range, sensitivity and spreading factor guide	https://zbotic.in/sx1276-lora-module-range-sensitivity-spreading-factor-guide/

11 Wokwi prototype pins

The node interface prototype (OLED screen, SOS button, RGB LED) runs in the Wokwi ESP32 simulator: <https://wokwi.com/projects/476841677745845249>. Pins from README section 6, checked against `emergency-mesh/wokwi_node/main.py`.

Component	ESP32 pin
OLED SSD1306, SCL / SDA	GPIO 22 / 21
SOS button (internal pull-up)	GPIO 14
RGB LED, red / green / blue	GPIO 27 / 32 / 33

In `main.py`: I2C SCL = GPIO 22, SDA = GPIO 21; button = GPIO 14; LED red / green / blue = GPIO 27 / 32 / 33.

12 Known limits

System

- Every home needs a unit with power during an outage, and it must be within ESP-NOW reach of a relay.
- Unused equipment decays; heartbeats and drills reduce this but don't remove it.

- Mass events cause heavy collisions and long waits for low-priority messages (Section 3.3).
- Indian rules limit each relay to transmitting 2.5% of the time. Our forwarding has every relay repeat every message, so a mass event (like the 500-SOS test) would exceed that. Normal use, mostly heartbeats, uses about 0.4% of each relay's time (our own calculation, Section 3.3).
- The home unit has no login (nobody can sign in mid-disaster), so false alarms are possible; rate limits reduce this.
- It only helps if a control room or rescue team runs a base station.

Simulation

- Range is a simple circle on flat ground: no terrain, buildings or fading beyond random loss.
- Range and loss come from published figures and assumptions (Section 4), not our own measurements.
- Battery use is not modelled.
- Home units and ESP-NOW are not in the Python simulator; it models relays and the base only. The demo's 200 m home unit reach is an assumption.

Not measured

- Radio range (LoRa and ESP-NOW) and battery life have not been measured on real hardware.

From README section 9.